



OrganoidRL-Stable: Master Roadmap (v2.1)

Project Goal: Build OrganoidEnv, the first OpenAI Gym-compatible environment for training biological Spiking Neural Networks (SNNs).

Philosophy: "Stability First, Intelligence Second."

Timeline: 16 Weeks (4 Months)



Phase 0: Setup & Architecture

1. Technical Stack

- **Physics Engine:** Brian2 (with C++ Standalone Mode for speed)
- **Interface:** Gymnasium (Standard RL API)
- **Glue:** NumPy & Matplotlib

2. Directory Structure

Create this exact folder layout immediately:

```
organoid_rl/
├── environment/
│   ├── __init__.py    # Gym registration
│   ├── core.py        # Main OrganoidEnv class (Gym Interface)
│   ├── neurons.py     # Izhikevich equations + Metabolic Governor
│   ├── stimulator.py  # The "God Mode" stabilizer logic
│   └── rewards.py     # Eligibility Trace logic
├── agents/
│   └── baseline_agent.py # Simple random/heuristic agent for testing
├── experiments/
│   ├── 01_hello_brain.py # Week 1 proof of life
│   ├── 02_seizure_test.py # Week 2 stability test
│   └── 03_pavlov_task.py  # Week 5 learning test
├── setup.py           # For pip installation
└── requirements.txt   # Dependencies
```

3. Requirements (requirements.txt)

```
brian2>=2.5.1
gymnasium>=0.29.0
numpy>=1.24.0
matplotlib>=3.7.0
scipy>=1.10.0
```



MONTH 1: The "Heartbeat" Phase (Environment Stability)

Goal: A stable simulation that runs for 1 hour without seizing (exploding) or dying (silence).

Week 1: The Metabolic Neuron (Physics Engine)

- **Day 1 (Setup & Verification): * Install Core Libraries:** Run `pip install -r requirements.txt`.
 - **Compiler Check:** Brian2's `cpp_standalone` mode requires a C++ compiler (Visual Studio Build Tools on Windows, Xcode on Mac, or `build-essential` on Linux). This is **non-negotiable**; standard Python is 50x too slow for this project.
 - **Package Structure:** Ensure `organoid_rl/setup.py` exists. Run `pip install -e .` in the root directory. This allows you to import `organoid_rl` anywhere, treating your project like a professional library (e.g., `gym`, `numpy`).
 - **Sanity Check:** Write a script that imports `brian2`, sets `prefs.codegen.target = 'cython'`, and runs a 10ms simulation. If it errors out, fix your C++ compiler path before moving forward.
- **Day 2 (The Metabolic Governor): * Concept:** Implement the "Metabolic Izhikevich" model in `environment/neurons.py`. Standard neuron models assume infinite energy.

Biological neurons run on ATP. We simulate this by adding a state variable E (Energy).

- **The Equation:** Add $dE/dt = (1 - E)/\tau_{recovery}$ to your equations.
 - E is a dimensionless variable (0.0 to 1.0).
 - 1.0 represents "Full ATP Reserves".
 - $\tau_{recovery}$ (approx 100ms) represents how fast the mitochondria generate new energy.
- **The Constraint:** Crucial step—modify the spike threshold logic. The neuron fires *only if* $v > 30mV$ **AND** $E > 0$.
- **The Cost:** On every spike, decrement energy: `E -= spike_cost` (e.g., 0.1). This creates a physical limit on firing rates.

- **Day 3 (The "Stress Test" Experiment): * Protocol:** Create experiments/01_stress_test.py. Inject a dangerously high current ($I > 200pA$) into a single neuron for 5 seconds.
 - **Expected Behavior:**
 1. **Phase 1 (Fresh):** Neuron fires rapidly (Gamma band, ~60Hz).
 2. **Phase 2 (Depletion):** Firing rate naturally slows down as E drops below equilibrium.
 3. **Phase 3 (Exhaustion):** The neuron goes **silent** or enters "bursting" mode despite the constant high input.
 - **Why this matters:** This proves your "Stabilizer" works. A standard Izhikevich neuron would fire at 60Hz forever, leading to the "Seizure" state. Your neuron self-regulates.
- **Day 4-5 (Synaptic Transmission & Delays): * Synapse Model:** Use Synapses(pre, post, model='...', on_pre='...').
 - **The Delay Factor:** You **must** add a transmission delay (synapses.delay = '1*ms + rand()*2*ms').
 - *Reason:* Real brains have axonal delays. Without this delay, STDP (Spike-Timing Dependent Plasticity) is mathematically impossible because the "cause" (pre-spike) and "effect" (post-spike) would happen at the exact same timestep.
 - **Voltage Jump (EPSP):** Define the on_pre action as $v_{post} += w$. This simulates the sudden influx of ions (EPSP) when a neurotransmitter hits the receptor.
 - **Verification:** Plot the membrane potential of the Post-neuron. You should see a sharp vertical jump 2ms *after* the Pre-neuron fired.
- **Week 1 Deliverable:** A "Metabolic Validation Plot" with three subplots sharing the X-axis (Time):
 1. **Top:** Input Current (Step function).
 2. **Middle:** Neuron Voltage (showing spikes -> silence).
 3. **Bottom:** Energy Level E (starting at 1.0, dropping to 0.0, and recovering).
 - *Significance:* This plot is your "Certificate of Authenticity" that your simulation respects biological constraints.

Week 2: The "Seizure" Test (Scaling Up)

- **Day 1 (Population):** Scale to 1000 neurons (800 Excitatory, 200 Inhibitory).
- **Day 2 (Wiring):** Random connectivity ($p = 0.1$). Set weights: $w_{exc} = 10mV$, $w_{inh} = -20mV$.
- **Day 3 (The Crash):** Run the simulation with random noise. It *will* likely seize (fire uncontrollably) or die.
- **Day 4 (Tuning):** Adjust tau_recovery (Energy recovery speed) and Inhibitory Weights.
 - *Goal:* The network should burst periodically (like brain waves) but not lock up.
- **Day 5 (Visualization):** Create a Raster Plot (Spike Time vs Neuron Index).
- **Week 2 Deliverable:** A 60-second simulation where the network stays active (~5-10Hz)

without turning into a solid block of spikes.

Week 3: The Interface (Gymnasium Integration)

- **Day 1 (Class Setup):** Fill in environment/core.py. Implement `__init__`, `reset`, `step`.
- **Day 2 (Action Space):** Map action (int 0-7) to `current_injection` into specific neuron groups.
 - *Logic:* Action 0 -> Stimulate Neurons 0-50. Action 1 -> Neurons 51-100.
- **Day 3 (Observation Space):** Implement "Spike Rate Coding".
 - *Logic:* Count spikes in the last 100ms for each group. Normalize to 0-1.
- **Day 4 (The "God Mode" Stabilizer):** Implement environment/stimulator.py.
 - *Logic:* In `step()`, check global spike count. If < 5 spikes, inject Poisson noise. If > 500, inject strong Inhibition.
- **Week 3 Deliverable:** A script that runs `env.step(env.action_space.sample())` loop and prints valid observations.

Week 4: The Memory (Eligibility Traces)

- **Day 1 (Trace Theory):** Implement traces in environment/rewards.py.
 - *Equation:* $dTrace/dt = -Trace/\tau_{trace}$.
- **Day 2 (Trace Events):** Modify Synapse equations: `on_pre='Trace += 1'`.
- **Day 3 (Weight Update):** Implement the "Dopamine" signal.
 - *Logic:* `w += learning_rate * Reward * Trace`.
- **Day 4-5 (Validation):** The "delayed reward" test.
 - Stimulate -> Wait 100ms -> Reward.
 - Check if weights increased.
- **Week 4 Deliverable:** A graph proving that weights increase even if Reward arrives *after* the spike.



MONTH 2: The "Pavlov" Phase (Conditioning)

Goal: Train the network to associate a specific input with a specific output.

Week 5-6: Task 1 - The Bell & Food

- **Task:** Stimulate Region A (Bell). Goal: Maximize firing in Region B (Saliva).
- **Action:** RL Agent picks which region to stimulate.
- **Reward:** +1 if Region B firing rate > Threshold.
- **Metric:** Measure "Reaction Time" (how fast Region B fires after Region A).
- **Deliverable:** Learning curve showing Success Rate going from 10% (random) to >80%.

Week 7-8: Negative Reinforcement

- **Task:** Stimulate Region A. Goal: FIRE Region B, but SILENCE Region C.
- **Reward:** +1 for Region B spikes, -5 for Region C spikes.
- **Why:** Proves the network can inhibit specific pathways.

- **Deliverable:** A plot showing Region C going silent over time.



MONTH 3: The "Intelligence" Phase

Goal: Solve a non-linear problem (Pattern Recognition).

Week 9-10: Task 2 - Pattern Recognition (XOR-like)

- **Task:**
 - Input Pattern 1 (Neurons 0-50 active) -> Output Region 1 must fire.
 - Input Pattern 2 (Neurons 51-100 active) -> Output Region 2 must fire.
- **Challenge:** The inputs overlap slightly. The network must distinguish them.
- **Deliverable:** Accuracy graph distinguishing Pattern 1 vs Pattern 2.

Week 11-12: Ablation Studies (The Science Part)

- **Experiment A:** Run Task 2 *without* Eligibility Traces. (Should fail).
- **Experiment B:** Run Task 2 *without* Metabolic Governor. (Should seize and fail).
- **Why:** This proves your components are *necessary*. This is what reviewers look for.



MONTH 4: Publication & Packaging

Goal: Polish the code and write the paper.

Week 13: Paper Writing

- **Title:** "OrganoidEnv: A Stabilized Reinforcement Learning Environment for Biological Spiking Networks."
- **Figures:** Generate clean matplotlib figures of the Raster Plots, Learning Curves, and Ablation comparison.

Week 14: Code Polish

- **Clean Up:** Remove commented-out code. Add docstrings.
- **Docker:** Create a Dockerfile so anyone can run it without installing Brian2 manually.
- **README:** Write a high-quality README with badges and installation instructions.

Week 15-16: Release & Outreach

- **GitHub:** Make repo public.
- **Submission:** Submit to *Frontiers in Neuroscience* or a Neuromorphic workshop.
- **Social:** Post the "Pong" or "Pattern Rec" video on Twitter/LinkedIn.



Critical Implementation Details

1. The Metabolic Governor Equation (Neurons)

Add this to your Brian2 equations string:

```

# v = membrane potential, E = energy
eqs = '''
dv/dt = (0.04*v**2 + 5*v + 140 - u + I) / ms : 1
du/dt = a*(b*v - u) / ms : 1
dE/dt = (1.0 - E) / 100*ms : 1 # Recovery tau = 100ms
I : 1
'''

# Reset condition logic:
# If E > 0: Fire normal spike, E -= 0.1
# If E <= 0: Do NOT fire (clamp v), E recovers slowly

```

2. The God Mode Logic (Stabilizer)

```

def check_stability(self):
    total_spikes = self.spike_monitor.count[-100:] # Last 100 steps
    if total_spikes < 5:
        self.input_group.I = 50 # Kickstart
    elif total_spikes > 500:
        self.input_group.I = -100 # Emergency Brake (Hyperpolarize)

```

3. Eligibility Trace Update (Rewards)

```

# On Pre-synaptic spike:
trace += 1.0

# On Global Reward Signal (End of Episode):
w += learning_rate * reward * trace

```